

[< back](#)

| Vol. 36, No. 02, pp. 311-341 (2026)

 Free Access

An Environment Adaptation Agent of Reinforcement Learning in Continuous Integration Test Case Prioritization

Zheng Li, Jiping Liu, Shunqing Xu, Hengyuan Liu, and Yong Liu

<https://doi.org/10.1142/S0218194025500792> | Cited by: 0 (Source: Crossref)[< Previous](#)

Abstract

Continuous Integration (CI) is a fundamental practice in modern software development. It enables early fault detection through regression testing, where Test Case Prioritization (TCP) plays a crucial role in improving detection efficiency. Reinforcement Learning-based TCP (RL-TCP) has shown promise in CI scenarios, but its performance often fluctuates due to CI's dynamic nature. Existing solutions address this by assigning additional rewards or periodically retraining agents. However, these methods either risk over-adjusting strategies due to excessive reliance on additional rewards or fail to respond promptly to changes due to fixed retraining intervals. We propose a novel Environment Adaptation Agent-based RL-TCP method (EAA) that addresses these challenges through a dual mechanism. EAA detects significant environmental changes by analyzing fluctuations in prioritization effectiveness. When a change is detected, it assigns targeted rewards to test cases. EAA also refines the agent's gradient update so that environmental dynamics are better incorporated into retraining. This enables agents to swiftly adapt while retaining learned prioritization knowledge. Evaluations on 12 real-world industrial datasets show that EAA improves the NAPFD metric by 4.7–24.79% and reduces the average TTF by 35.85–50.37 positions compared to state-of-the-art RL-TCP methods. Additionally, EAA significantly reduces occurrences of NAPFD equal to zero, effectively mitigating prioritization instability.

1. Introduction

Continuous Integration (CI) has become a widely adopted core practice in modern software development. Its key principle is the rapid and frequent integration of decentralized development work into a shared mainline, using automated builds and regression testing to detect potential faults early [1]. Unlike traditional centralized development models, CI requires developers to frequently integrate code within short cycles. This high-frequency integration effectively reduces the conflicts and risks associated with large-scale merges in later stages while providing more timely fault feedback through automated testing. In this process, Test Case Prioritization (TCP) techniques can effectively reduce the cost of regression testing [2]. With the frequent updates to code and test cases in CI environments, along with the increasing scale and complexity of software, traditional regression testing-oriented TCP techniques are becoming less suitable [3]. Consequently, a growing number of studies are applying machine learning and Reinforcement Learning (RL) methods to the TCP problem in CI environments. Reinforcement Learning-based Test Case Prioritization (RL-TCP) approaches learn the optimal policy by continuously interacting with the environment, allowing for incremental updates to new data [4]. The iterative update nature of RL makes RL-TCP especially advantageous in handling the complex, rapidly evolving scenarios of CI.

RL, as a data-driven approach, heavily relies on the stability of data. In actual CI processes, the evolving requirements often trigger adjustments to testing objectives, leading to the continuous evolution of the test suite, including the addition, modification and deletion of test cases. In RL, the dynamic changes in CI environments can hinder the agent's effective learning of prioritization knowledge [5]. Moreover, subsequent updates may lead to excessive adjustments of the prioritization strategy, weakening previously learned knowledge and degrading overall prioritization performance. These factors ultimately lead to fluctuations in TCP results. They make it difficult to maintain consistently high prioritization performance and may occasionally result in severely degraded outcomes. Such instability can negatively impact fault detection efficiency and delay the feedback cycle in CI workflows.

To address the above challenges, previous research has introduced improvements to the two core components of RL: reward and agent [6]. On the one hand, additional rewards are

[< back](#)

efficiently [7]. On the other hand, researchers retrain the agent after fixed intervals of CI cycles to better adapt to the changes in the CI environment [4]. Although these methods have achieved positive results to a certain extent, some limitations remain. First, continuously providing additional rewards to test cases can guide the agent toward learning a better prioritization strategy. However, this may also cause excessive reliance on the additional reward signal, resulting in over-adjustment of the prioritization strategy and reduced stability. Second, retraining at fixed intervals can ensure the continuous update of the agent. However, this approach responds insufficiently promptly to sudden environmental changes, as fixed intervals introduce a certain degree of delay [8]. Most importantly, these methods do not incorporate environmental change factors into the agent's internal training process. They also fail to ensure that the agent retains previously learned knowledge while adapting to new environmental changes. As a result, they cannot effectively mitigate fluctuations in prioritization performance.

To address these limitations, we primarily focus on incorporating environmental change factors into the agent's internal training process and propose a novel RL-TCP method based on the Environment Adaptation Agent (EAA), consisting of two main parts: "Environment Change Detection and Additional Reward Triggering" and "Reconciliation Method".

Specifically, in the first part, we assign additional rewards to test cases in the current CI cycle only when a significant environmental change is detected. By further assigning higher rewards to test cases that are more likely to fail under varying conditions, the agent is guided to prioritize these test cases earlier in the ordering process. Thus, the additional rewards are also treated as a reflection of environmental changes and are integrated into the agent's retraining process. This approach dynamically assigns additional rewards based on environmental changes. It reduces the frequency of assigning additional rewards to test cases, but ensures that the rewards are more precise and effective in addressing environmental changes. By promptly detecting changes in the CI environment and immediately retraining the agent, the method ensures that the additional rewards effectively guide timely updates to the agent's prioritization strategy. Through the above operations, CI change detection and additional rewards are incorporated into the agent's training process as factors representing environmental changes.

Furthermore, in the second part, during the agent retraining process, we integrate a reconciliation method to constrain the gradient updates of the agent model. This method

[< back](#)

directions that have less influence on the original strategy. It aims to prevent excessive adjustments to the original strategy when the agent updates the prioritization strategy. This ensures better integration between new prioritization knowledge derived from environmental changes and the knowledge previously learned. Ultimately, by integrating the two modules, EAA can better adapt to changes in the CI environment. It mitigates fluctuations in prioritization performance and enhances both the accuracy and overall effectiveness of TCP.

We conducted experimental evaluations of EAA on 12 real industrial datasets. Compared to the best existing RL-TCP methods, the proposed method improves NAPFD by 4.7–24.79%, decreases TTF by approximately 35.85–50.37 positions, and significantly reduces the number of CI cycles with NAPFD equal to zero. Moreover, the experiments validate that each component of EAA contributes meaningfully to improving prioritization performance and enhancing prioritization stability.

The main contributions of this paper can be summarized as follows:

- We propose a novel RL-TCP method EAA which guides retraining by detecting changes in the CI environment. By integrating environmental changes into agent retraining, EAA mitigates fluctuations in prioritization performance.
- We design an additional reward allocation method based on CI environment changes. When a significant change in the environment is detected during the current CI cycle, the method assigns additional rewards to test cases.
- We incorporated a reconciliation method into the agent's training process. The method projects the updates from subsequent prioritization tasks onto a subspace that does not interfere with the gradient directions learned by the agent from previous tasks. This method helps to better integrate new prioritization knowledge brought about by environmental changes with previously learned prioritization knowledge.
- We conducted experiments on 12 actual industrial datasets, and the results demonstrate the effectiveness and broad applicability of the method. We shared the source code and datasets for the study in the GitHub repository.^a

[< back](#)

testing and CI testing methods for TCP, as well as the progress in research on RL-TCP. Section 3 provides a detailed explanation of the proposed method. Section 4 describes the overall setup and implementation process of the experiments. Section 5 conducts a thorough analysis of the method's effectiveness. Section 6 discusses potential threats to the study and their impacts. Section 7 concludes the paper and presents corresponding conclusions. Finally, Sec. 8 outlines future work directions.

2. Background and Related Work

2.1. Test case prioritization

2.1.1. Test case prioritization in traditional regression testing

TCP [9] aim to improve the efficiency of software development by executing test cases that are more likely to detect errors as early as possible. Traditional TCP methods typically rely on heuristic policies to prioritize test cases by leveraging information such as code coverage, historical execution records and requirements [10].

Earlier studies determined TCP based on code coverage [11], and these approaches significantly improved fault detection rate. Subsequent research gradually focused on historical information, utilizing historical failure data and execution records of test cases for prioritization [12–14]. Search-based TCP approaches employ search policies to find optimal or near-optimal orderings within the global space of single-objective or multi-objective prioritization sequences [15–18].

With the evolution of software development practices, especially the widespread adoption of CI, traditional TCP methods in regression testing face numerous new challenges. In CI environments, frequent code changes, large and dynamically evolving test suites pose significant challenges. Consequently, traditional TCP methods require further improvements to adapt to these rapidly changing testing demands.

2.1.2. Test case prioritization in continuous integration testing

CI has become a widely adopted software development practice, allowing team members to frequently merge new code into the codebase during the development process. These frequent

[< back](#)

rapid feedback, TCP often needs to be performed in time-constrained environments, making it necessary to select only parts of test cases for execution [4, 19].

With the rapid development of machine learning, more and more TCP methods are incorporating machine learning techniques, leading to better results [20, 21]. Machine learning-based methods can generally be classified into four categories [22]: ranking-based models [23, 24], clustering-based models [25, 26], Natural Language Processing (NLP)-based models [27, 28] and RL-based models [4, 29].

Lachmann *et al.* [23] proposed a method using the machine learning algorithm SVM Rank, which prioritizes test cases by considering historical information. Khalid and Qamar [30] employed the K-Means clustering method, combining customer allocation weights abstracted from business requirements to prioritize customer preferences. Sutar *et al.* [27] introduced a method that integrates NLP techniques to identify high-potential regression test cases by intentionally matching them with defects in the main test suite. Yaraghi *et al.* [31] defined a comprehensive set of features commonly used in related research and captured the relationships between data through defined data models. Their research demonstrated that, among many features, Test Case Execution Record Features (REC) contain highly effective information that can support more efficient fault localization.

Spieker *et al.* [4] introduced the RETECS framework. The successful exploration of RETECS demonstrated the immense potential of applying RL methods to the TCP problem, showcasing its advantages in improving testing efficiency and fault detection rates.

In recent years, numerous researchers have explored the integration of RL. Schwendner *et al.* [32] introduced a pipeline-aware regression testing optimization strategy that employs RL models to distinguish testing objectives between the pre-submit and post-submit phases. Moreover, Mhalla and Saied [33] presented a decision tree classifier based on deep RL to accurately identify commits that should be skipped in CI execution. In addition, hybrid strategies combining meta-heuristic optimization techniques with RL have been proposed, integrating heuristic search with agents [34, 35].

2.1.3. Reinforcement learning-based test case prioritization for continuous integration

< back

particularly well-suited for addressing the TCP problem in CI environments [21].

Spieker *et al.* first attempted to apply RL to the CI TCP problem and proposed the RETECS framework (see Fig. 1). The study used both table-based agent models and neural network-based agent models and proposed three different reward functions. Among these models, the combination of the neural network agent model and the Test Case Failure (TCF) reward function achieved the best performance. RETECS adopts an experience replay mechanism [36] to periodically retrain the agent.

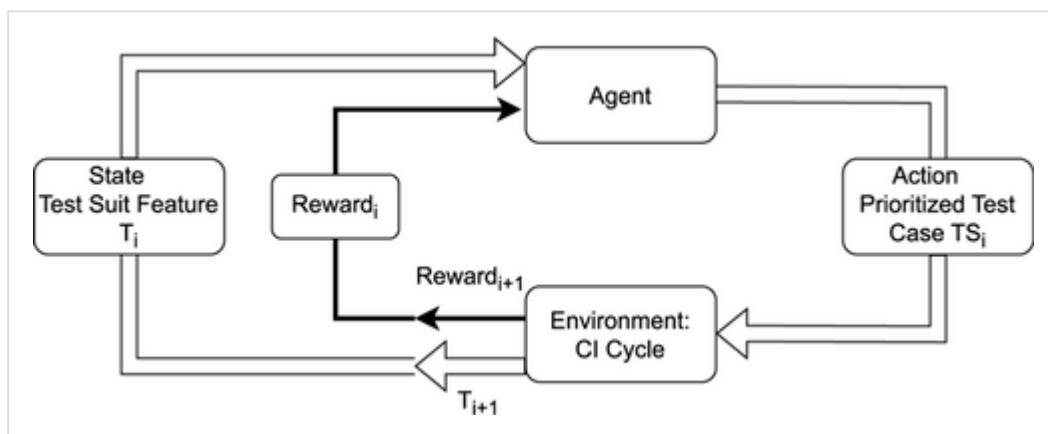


Fig. 1. The RETECS framework for TCP based on RL.

With the introduction of RETECS, many RL-TCP methods have emerged. Early research primarily focused on the design of reward functions and proposed several improved versions of reward functions based on the RETECS framework [19, 37, 38]. For example, He *et al.* [39] considered the historical failure distribution of test cases and proposed the Average Percentage of Historical Failure (APHF) reward function. Wu *et al.* [40] proposed a time-window-based reward function that extracts historical execution information using window size to prioritize test cases in a CI environment.

In subsequent research, Bertolino *et al.* [41] further validated the effectiveness of RL in CI testing. Chen *et al.* [7] designed a rule-based agent with an additional reward mechanism aimed at giving more attention to newly added test cases to mitigate performance fluctuations and improve the accuracy of prioritization. Bagherzadeh *et al.* [29] combined search methods from the information retrieval domain with popular RL algorithms and conducted extensive experiments [42, 43]. Qian *et al.* [44] proposed a RL test case prioritization method (TCP-

[< back](#)

which enhances the performance of TCP in a CI environment by improving the initial learning environment, designing a comprehensive reward metric and adaptive fine-tuning strategies. The recently proposed Attention Transfer Reinforcement Learning approach (ATRL-TCP) adapts its input representation according to dataset characteristics by leveraging historical information and error rates, thereby improving the generalization of TCP in CI [45]. In terms of RL interpretability, some work has examined the interpretability of LTR models within TCP, introducing various explanation scenarios and providing experimental analyses [46]. Meanwhile, specific studies have explored AutoML approaches that focus on automatic model selection and hyperparameter tuning, with the goal of lowering the barrier to model adoption [47, 48].

2.2. Gradient projection in continual learning

Continual Learning (CL) aims to enable models to learn a sequence of tasks without forgetting previously acquired knowledge [49]. However, a major challenge in CL is the catastrophic forgetting problem, where learning new tasks causes a significant degradation of performance on earlier tasks [50]. To address this, the Gradient Projection approach has been widely adopted as an effective solution [41]. The core idea behind this method is to constrain the gradient update for a new task to directions orthogonal to the gradient subspace of previous tasks, thereby minimizing interference with the knowledge already learned and helping the model retain past information while accommodating new data.

One of the pioneering methods in this domain is Gradient Episodic Memory (GEM), which adjusts the gradient direction to prevent increases in loss on previous tasks [51]. Averaged GEM (A-GEM) further optimizes this by simplifying the projection process, reducing computational overhead [52].

Based on these concepts, the Gradient Projection Memory (GPM) method was proposed [53]. GPM identifies the significant subspaces of each task by performing a Singular Value Decomposition (SVD) of the activation space of each task. When training on a new task, the gradients are projected to locations orthogonal to these subspaces, thus effectively retaining prior knowledge. This approach improves the learning ability of the new task while maintaining the performance of the old task.

[< back](#)

(TRGP) selects relevant past tasks based on task similarity, enabling more flexible gradient updates within a defined trust region [54].

In addition, the Prompt Gradient Projection (PGP) method combines gradient projection with Prompt Tuning, which utilizes the instance query mechanism of Prompt Tuning to further reduce the interference with old tasks [55]. The method effectively prevents forgetting in the Vision Transformer (VT) architecture by constructing the gradient projection matrix through SVD of the element-level summation space of the input space and the cue space.

These developments highlight the effectiveness of gradient projection methods in mitigating catastrophic forgetting. By strategically constraining gradient updates, these approaches enable models to retain prior knowledge while acquiring new information, striking a balance between stability and adaptability in CL scenarios.

3. Method

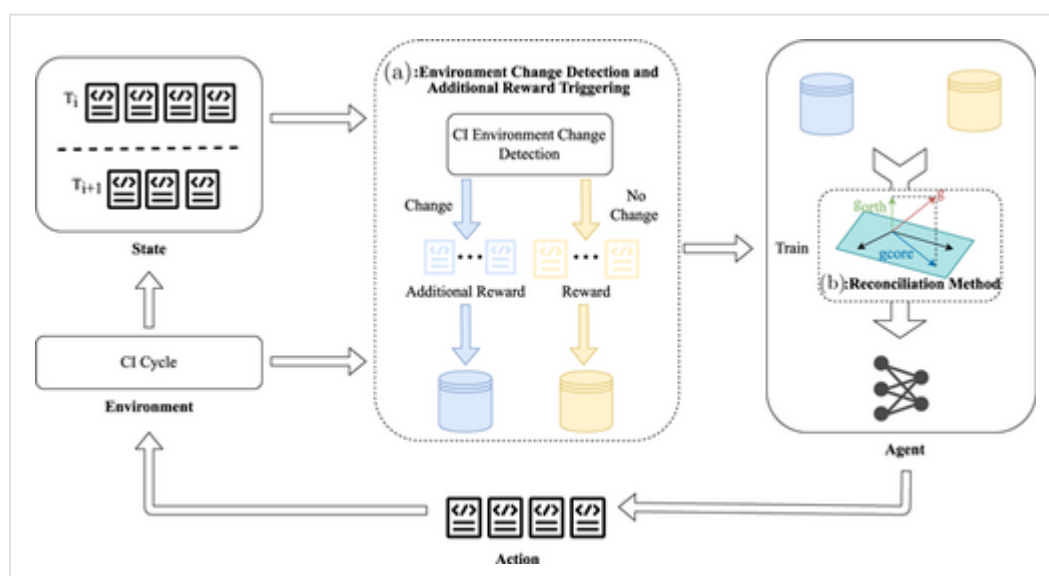
Previous research has aimed to mitigate performance fluctuations in TCP by improving the reward mechanism and the agent. Some studies continuously assign additional rewards to test cases during prioritization to facilitate the agent's acquisition of prioritization knowledge [7]. However, continuously providing additional rewards may make the agent overly dependent on the guidance from these reward signals, causing excessive adjustments in prioritization strategy updates. Such over-adjustments can result in forgetting previously learned prioritization knowledge, thus reducing the stability of prioritization performance. In addition, some studies perform agent retraining at fixed CI cycle intervals to maintain continuous updates, allowing the agent to adapt to changes in the CI environment [4]. Since the timing of changes in the CI environment is uncertain, periodic retraining may cause a certain degree of delay. This delay makes it difficult to respond flexibly to changes in the environment and to adjust the prioritization strategy promptly. Although these methods help alleviate the performance fluctuation problem, none of them involves the internal improvement of the agent and fails to ensure that the agent effectively retains the previously learned knowledge while adapting to the environmental changes.

[< back](#)

factors into the agent's training process. Specifically, the method detects CI environment changes by analyzing TCP outcomes to respond to the uncertainty of environmental changes. Upon detecting significant changes, the method assigns additional rewards to test cases as a representation of CI environment changes and immediately retrain the agent. Furthermore, we incorporate a reconciliation method to integrate environmental change factors into the retraining process. This method is designed to guide updates to the agent's prioritization strategy through new rewards while preserving the prioritization knowledge already learned by the original strategy, thereby facilitating the integration of new and existing knowledge.

In brief, unlike existing RL-TCP methods that either rely on continuous additional rewards or periodic retraining, EAA explicitly incorporates CI environment changes into the agent's internal training process. This addresses three existing problems in current RL-TCP research: (1) the lack of timely adaptation to sudden environment changes, (2) the absence of a more adaptive and precise additional reward mechanisms and (3) the inability to retain previously learned prioritization knowledge while adapting to new conditions.

As shown in Fig. 2, the two main components of the framework based on EAA are as follows: (a) Environment Change Detection and Additional Reward Triggering and (b) Reconciliation Method. The overall process of the framework involves feeding test case data from each cycle as state inputs into the agent. The agent then assigns a priority to each test case as an action based on the existing prioritization strategy. The environment responds to these actions by providing feedback in the form of reward values.



[< back](#)

Specifically, in each cycle, EAA assesses the extent of changes in the CI cycle based on the prioritization feedback from the current and previous cycles. If significant changes are detected, the method performs experience replay [36] immediately, using both accumulated experience from previous cycles and additional experience, rather than waiting for fixed cycle intervals. These experiences consist of test case features and their corresponding rewards. Furthermore, EAA incorporates a reconciliation method during training to more effectively integrate environmental changes into the agent's retraining process. Subsequently, the CI environment provides corresponding rewards, generating new experience. If significant changes in the CI cycle are detected, additional rewards are added to test cases, generating additional experience. Although we no longer continuously add additional rewards to test cases, timely experience replay ensures that these rewards are promptly used to guide the agent. Through these measures, EAA ensures that the agent is promptly updated to adapt to changes in the CI environment. At the same time, it mitigates the impact of updates on previously learned knowledge. As a result, the performance of test case prioritization is further enhanced.

3.1. Environment change detection and additional reward triggering

Previous studies retrain the agent at fixed CI cycle intervals, but the uncertainty of environmental changes often results in delayed updates, which prevents the prioritization strategy from being adjusted promptly. In addition, in previous research, the additional reward mechanism consistently adds additional reward to test cases. In the long-term process of TCP, there can be instances where the agent becomes overly dependent on the guidance of additional rewards. Consequently, the priority strategy might undergo excessive adjustments, undermining the learned knowledge. Ultimately, these issues degrade the overall performance of the TCP strategy, resulting in performance fluctuations.

To address these issues, we designed a component to detect the degree of change in CI cycles and modified the additional reward mechanism for test cases. For each cycle t , we record the Normalized Average Percentage of Faults Detected (NAPFD) metric as N_t . We divide the data into a current observation window W_c and a historical observation window W_h :

[< back](#)

(1)

$$W_h = \{N_{t-w_c-w_h+1}, \dots, N_{t-w_c}\}, \quad |W_h| = w_h.$$

(2)

We apply the Mann–Whitney U test on W_c and W_h to assess the significance of changes.

Denote the resulting p -value as p_t :

$$p_t = \text{Mann – Whitney } U(W_c, W_h).$$

(3)

We adopt the significance level $\alpha = 0.05$, a widely accepted convention in statistics and commonly used in software engineering and machine learning research to determine statistical significance [56]. If $p_t < \alpha$ or $N_t = 0$, we consider a significant change in the CI environment has occurred at cycle t . The NAPFD metric is used to evaluate the effectiveness of prioritization [57]. A significant change in NAPFD indicates a deterioration in the accuracy and effectiveness of the agent's prioritization strategy. In particular, when the NAPFD drops to zero, it is regarded as a critical failure of the prioritization policy.

We leverage the environment change assessment component to evaluate environmental variations across 12 datasets. The detailed results are shown in Fig. 3. The figure presents the p -values calculated for all CI cycles in which failing test cases were present. The red dashed line represents the significance threshold of 0.05. The results indicate that environmental changes occur to varying degrees across all datasets, and statistically significant changes are observed in nearly all of them.

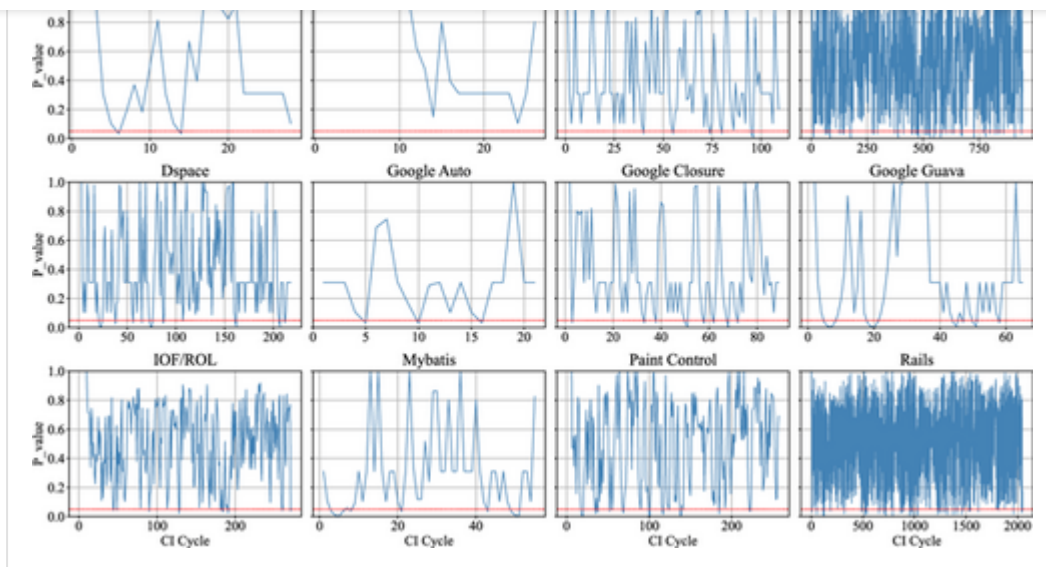
[< back](#)

Fig. 3. Significant CI environment change statistics.

When a significant change in the CI cycle is detected, the method assigns additional rewards to test cases, allowing failed test cases to receive higher rewards. This reflects the environmental changes by emphasizing the test cases that are more likely to fail under different conditions. At the end of the cycle, we store the experiences formed by test cases along with their corresponding additional rewards, which we refer to as additional experience. Moreover, the method immediately employs accumulated experience and additional experience for experience replay to retrain the agent rather than waiting for the next scheduled retraining period. Although the additional reward assignment method based on CI environment change detection reduces the frequency of adding additional rewards to test cases, the immediate experience replay ensures that these rewards are promptly utilized to guide the prioritization strategy update.

Overall, the method assigns additional rewards to test cases only when a statistically significant change is observed in the CI cycle and immediately retrains the agent. This method aims to mitigate prioritization performance fluctuations from the perspective of environment change detection and reward mechanism, thereby improving the accuracy of the prioritization strategy.

3.2. Reconciliation method for agent training

In the proposed method, an additional reward is assigned to test cases whenever a significant change occurs in the CI cycles. Subsequently, the method immediately performs experience

[< back](#)

mechanism to better integrate additional experience with historical experience. This design helps mitigate the impact of updates on previously learned knowledge. The reconciliation method is based on gradient projection technology. Its core idea is to project updates induced by additional rewards onto gradient directions that are both aligned with and orthogonal to the important gradient space of the past prioritization task knowledge. Of course, this projection mechanism applies similarly to basic rewards. This design helps balance the integration of new information with the retention of previously learned strategies, thereby enhancing the stability of the learning process. Specifically, this method analyzes the gradient spaces from past prioritization tasks to identify key gradient directions and then projects the updates onto these key directions and their orthogonal directions. This approach effectively integrates new information that reflects environmental changes into strategy updates. At the same time, it preserves the knowledge gained from the prioritization task and prevents excessive adjustments to the priority strategy, which could otherwise cause performance fluctuations in TCP. The so-called “important gradient space” refers to TCP task-critical knowledge information stored in vector form, where the directions of these vectors are closely related to the key features and knowledge of the task. A detailed explanation of this method is provided as follows.

For each CI cycle, we follow two main steps: identifying the basis of the important gradient space and calculating the importance of each basis vector. For illustrative purposes, we assume that a significant change is detected at the beginning of the $N + 1$ th cycle by analyzing the previous prioritization results and then retraining the agent. Each CI cycle performs TCP after the following steps, which we omit in the following presentation.

The N th CI cycle:

Find the space of important gradient: Passing the training data through the agent’s network, for each layer of the network, we get a representation matrix T of test case feature. T is a two-dimensional matrix with dimensions corresponding to the number of hidden layers and the number of input data samples. Perform SVD on matrix T , thus extracting the features, yielding $T = U\Sigma V^T$, where U and V are orthonormal matrices, and Σ is a diagonal matrix with singular values $(\sigma_i, 1)$ arranged in descending order along its diagonal, representing the importance of data in the corresponding directions. Subsequently, a norm-based criterion for k -rank approximation is applied to reduce the rank of the feature matrix, simplifying the problem

[< back](#)

are then selected, corresponding to spanning the important gradient space for the current cycle, thereby capturing the critical knowledge. Store these basis vectors in $M = [u_1, \dots, u_k]$, referred to as GPM.

Calculate the importance of each basis vector: Extract the singular value vector $\sigma = \text{diag}(\Sigma, k)$ corresponding to the first k basis in U from Σ . Then, for the i th basis vector, compute its importance λ_i by the following formula :

$$\lambda_i = \frac{(\alpha + 1)\sigma_i}{\alpha\sigma_i + \max(\sigma)},$$

(4)

where α is a non-negative hyperparameter. The basis vector associated with the largest singular value is assigned the maximum importance, while the importance of other basis vectors is assigned relative to this maximum value. The importance vector λ of the current cycle is $\lambda = [\lambda_1, \dots, \lambda_k]^T$. $\lambda_i = 1$ implies that the new task does not allow gradient steps along the corresponding basis direction, while gradients along other basis directions are scaled by a factor $(1 - \lambda_i)$.

The $N + 1$ th CI cycle:

Training the agent: In subsequent training, we impose restrictions on the new gradient update, $\nabla_W \text{Loss}$ as follows :

$$\nabla_W \text{Loss} = \nabla_W \text{Loss} - (M\Lambda M^T)(\nabla_W \text{Loss}),$$

(5)

where Loss denotes the loss of the current prioritization task. $\nabla_W \text{Loss}$ represents the gradient of the loss Loss with respect to the agent's model parameter W . This process involves calculating the importance matrix Λ , a diagonal matrix of size $k \times k$ where the diagonal entries contain the importance bases λ computed from previous tasks. Here, M is the matrix that stores the important gradient space bases from previous tasks. This gradient projection ensures that the gradient components along the directions orthogonal to M remain unchanged, while the

[< back](#)

stored in λ , following the scaling factor $(1 - \lambda_i)$.

Update the space of important gradients: After each cycle prioritization is completed, it is necessary to update the stored important gradient space M . First, construct the representation matrix T using data from the current cycle. Additionally, there may be basis directions in M that are either derived from the T via SVD or linear combinations of the bases in M . Therefore, to remove redundant bases and obtain a new representation matrix T' , the following steps are performed :

$$T' = T - (MM^T)T.$$

(6)

Next, perform SVD on T' and select the new k bases that satisfy the following conditions :

$$\|(MM^T)T\|_F^2 + \|(T')_k\|_F^2 \geq \epsilon_{th} \|T\|_F^2.$$

(7)

Then, add these new bases to M and update it accordingly.

Update the importance of each basis vector: The key to this step lies in updating the importance of the newly added k' basis vectors and the existing k basis vectors. Since T' is a partially processed representation matrix, performing SVD on it cannot fully capture the singular value distribution of T . Therefore, the importance of the newly added basis vectors is directly calculated from their corresponding singular values, while the importance of the existing basis vectors needs to be updated through an indirect method.

The following method can be used to update importance without relying on data from previous tasks. Perform SVD on $T_M = (MM^T)T = U_M \Sigma_M V_M^T$. According to Eq. (6), the first k column vectors of U_M are linear combinations of the basis vectors in matrix M . Thus, we obtain a singular value vector $\sigma_M = \text{diag}(\Sigma_M, k)$, where each element represents the importance of the corresponding basis vector in U_M for the current task. Since these redundant

[< back](#)

vectors in M using the following projection coefficient matrix :

$$C = M^T U_M.$$

(8)

Next, compute the singular value vectors of the old bases in M :

$$\sigma'_M = ((C \odot C)(\sigma_M)^2)^{1/2},$$

(9)

where $\odot$ denotes element-wise multiplication. For the new basis vectors, we perform SVD on T' to obtain the singular value vector for the k' newly added basis vectors σ'' . By merging the new and old singular value vectors, we ultimately obtain the complete singular value vector σ :

$$\sigma_\tau = \begin{bmatrix} \sigma'_M \\ \sigma'' \\ \sigma \end{bmatrix}.$$

(10)

The construction of σ as described above ensures the satisfaction of Eq. (7).

Using Eq. (4), the importance vector λ' for the basis vectors of the current task is then calculated. The cumulative importance for the task is computed, and the cumulative importance vector λ is updated. For $i \in [1, k]$, the importance of the old k basis vectors is updated as follows :

$$\lambda_i = \begin{cases} 1, & \text{if } (\lambda_i + \lambda_{i,old}) \geq 1, \\ \lambda_i + \lambda_{i,old}, & \text{otherwise.} \end{cases}$$

(11)

When the importance reaches 1, it signifies that the corresponding basis has been identified as critical for past tasks and will no longer be allowed to undergo gradient updates in future tasks.

[< back](#)

$\lambda = [\lambda^I, \lambda_{k+1}, \dots, \lambda_{k+k'}]^T$ is obtained. Additionally, update the total number of basis vectors as $k = k + k'$. These steps are then repeated for subsequent CI cycles.

4. Experimental Design

4.1. Research questions

RQ1: How does the proposed method perform in terms of CI TCP effectiveness and mitigating performance fluctuations? This RQ aims to compare the prioritization performance of the proposed method with existing TCP approaches and to evaluate its capability in mitigating performance fluctuations.

RQ2: How do the different parts of the proposed method affect the performance and effectiveness of the prioritization policy? In this RQ, we will conduct a detailed analysis of how each component of the proposed method contributes to these impacts.

4.2. Experimental subjects

To evaluate its effectiveness in real-world applications, we refer to previous research on CI-TCP and conduct experiments on 12 industrial datasets [7, 19]. These datasets have been adopted by multiple prior CI-TCP studies, which makes them well-established benchmarks for fair and reproducible evaluation. These datasets contain detailed information on test execution and historical results, and are well suited for the study of TCP.

Furthermore, these datasets include small-scale projects with only a few dozen test cases as well as large systems with thousands of test cases. They also cover hundreds to thousands of CI cycles, which allows the simulation of both short-term and long-term maintenance scenarios. In addition, the datasets contain projects with both low and high failure rates. Such diversity is beneficial for validating the robustness of the proposed method across different scales and fault characteristics.

The statistical information of the 12 datasets is shown in Table 1. For a more intuitive illustration of these characteristics, we further provide the corresponding Fig. 4. The definitions of the column names are as follows: “Test Cases” denotes the count of test cases; “CI Cycles” refers to the number of CI cycles; “Results” indicates the total execution results from the test

[< back](#)

proportion of test cases that have failed. From the information in the table, it can be seen that although the number of test cases varies from dozens to thousands in different datasets, the number of test cases in most datasets is more than 100. Meanwhile, the number of CI cycles in all datasets exceeds 300, and the number of CI cycles in some datasets exceeds thousands.

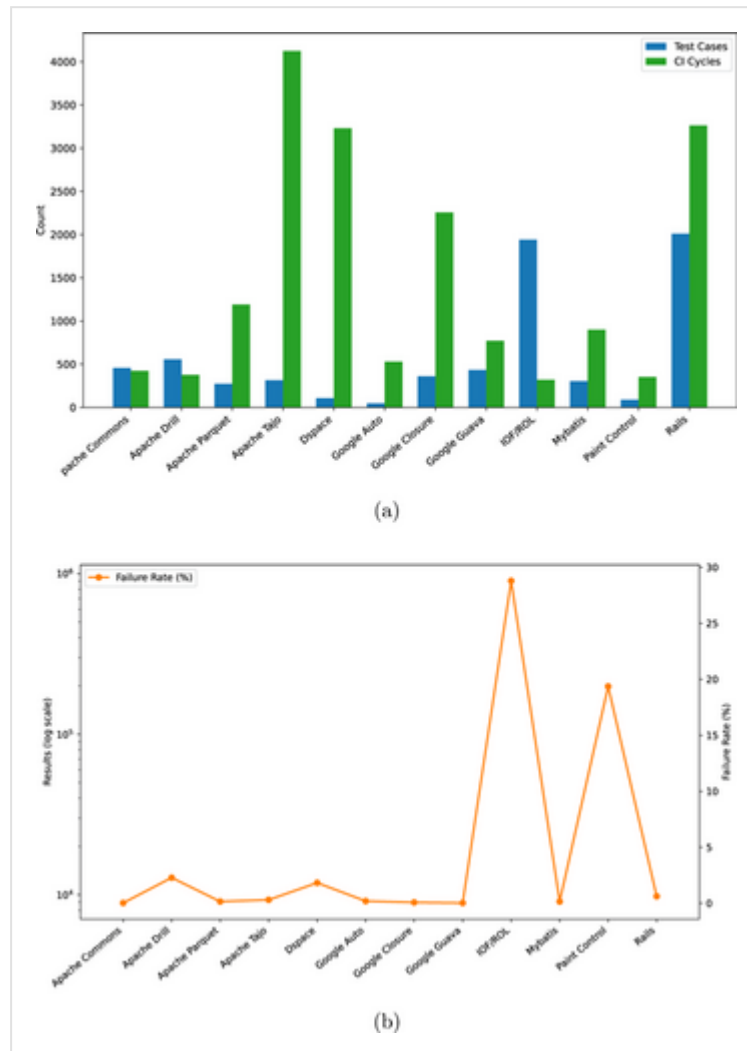


Fig. 4. Visual overview of the 12 datasets.

Table 1. Statistical information of 12 datasets.

Dataset	Test cases	CI cycles	Results	Failure rate
Apache Commons	457	423	332,650	0.02%
Apache Drill	556	376	8887	2.28%
Apache Parquet	273	1190	205,770	0.15%
Apache Tajo	314	4125	903,301	0.30%
Dspace	106	3230	204,161	1.82%

[< back](#)

Google Auto	46	530	14,601	0.19%
Google Closure	360	2257	663,470	0.07%
Google Guava	433	772	877,466	0.02%
IOF/ROL	1941	320	32,260	28.79%
Mybatis	303	899	815,598	0.15%
Paint Control	89	352	25,594	19.36%
Rails	2010	3263	781,273	0.62%

4.3. Measurement

In order to comprehensively evaluate the performance of the proposed method in this paper, we use some commonly used metrics for evaluation.

NAPFD (Normalized Average Percentage of Faults Detected) [57]: Since the maximum execution time is set for each CI cycle, it cannot execute all test cases. For this reason, we have adopted NAPFD as an evaluation criterion, which is capable of evaluating the test cases without executing all of them. The formula for NAPFD is as follows :

$$\text{NAPFD}(TS_i) = p - \frac{\sum_{j \in TS_i^{fail}} \text{Rank}(j)}{|TS_i^{fail}| \cdot |TS_i|} + \frac{p}{2 \cdot |TS_i|},$$

(12)

where

$$p = \frac{|TS_i^{fail}|}{|TS_i^{total, fail}|}.$$

(13)

$|TS_i|$ denotes the total number of test cases to be executed in the selected subset TS_i , where TS_i is a subset of the entire test suite TS . $\text{Rank}(j)$ denotes the position of the j th failed test case in the ordered test case sequence TS_i , where $|TS_i^{fail}|$ denotes the number of failed test cases detected in the test case suite TS_i and $|TS_i^{total, fail}|$ denotes the number of all failed test cases in the test case suite TS_i . NAPFD has a value between 0 and 1, with larger values indicating better prioritization results.

[< back](#)

shown in Eq. (13). The Recall value also ranges from 0 to 1, with larger values indicating that more failed test cases can be detected in a limited execution time.

TTF (Test to Fail): TTF is the average position of the first failed test case in the test suite. A smaller value means that the first failed test case detected is positioned farther forward and errors can be detected faster.

Duration: Duration refers to the average time required for the algorithmic framework. A shorter duration means faster execution.

4.4. Implementation

In the following methods, we conduct 30 independent repeated experiments and take the average as the final experimental result to reduce the impact of randomness on the outcomes effectively. In the proposed method, we use the metadata of test cases (sourced from RETECS) along with the execution results from the most recent seven historical cycles as input features. Furthermore, the experimental context is set in a time-constrained CI TCP scenario. Similar to previous research, only the first 50% of the prioritized test cases duration is executed within each CI cycle. Unlike previous research, our experimental analysis considers only CI cycles in which failed test cases are present.

Baselines: FoNT [7], a RL-based Continuous Integration Test Case Prioritization (CI-TCP) technique, utilizes expert knowledge via predefined rules and adds additional rewards to new test cases, thereby enhancing attention to new test cases. FoNT is also an effective method for mitigating prioritization performance fluctuations. We also considered the method RETECS [4], which is also a CI-TCP technique. In addition, we also considered the work of Yaraghi *et al.* [31]. However, since the dataset used in this paper cannot obtain all the features defined in the original paper, the labels required for training cannot be obtained either. To address this limitation, we made modifications to the Scalable approach. Specifically, this study adopts the RETECS framework as the base, utilizes REC features as the input to the agent, and introduces TCF as the reward function. The resulting method is referred to as RL-REC. This method, enriched with comprehensive environmental feature information, holds unique value in exploring the adaptability of feature engineering to the dynamic nature of CI environments.

[< back](#)

the existing feature set. The inputs for the other methods are kept consistent with EAA.

Table 2 provides an overview of the comparison between EAA and the other three baseline methods. As shown, existing approaches often overlook environmental changes and suffer from shortcomings in retraining and additional reward mechanisms. In contrast, EAA introduces novel strategies specifically designed to address these issues. As a result, it demonstrates stronger capability in mitigating prioritization instability within dynamic CI environments.

Table 2. Comparison between EAA and baseline methods.

Method	RETECS	FoNT	RL-REC	EAA
Environmental factors	✗	✗	✗	✓
Adaptive additional reward	—	✗	—	✓
Timely retrain	✗	✗	✗	✓
Knowledge reconciliation	✗	✗	✗	✓

The design of the reward function ATCF in FoNT is illustrated in the following Eq. (14). The reward function for RETECS and RL-REC is shown in Eq. (15). In this paper, EAA uses Average Percentage of Historical Failures (APHF) [39] as the base reward function and uses ATCF as an additional reward for test cases. This design choice is motivated by the intention to assign higher rewards to test cases that are more likely to fail, through the combined use of APHF and ATCF as dual reward signals. Ultimately, this approach aims to place greater emphasis on test cases with a higher likelihood of failure. By doing so, the agent is guided to prioritize fault-revealing test cases, thereby improving the overall effectiveness of the prioritization strategy. It is worth noting that, to ensure consistency and comparability across results from different reward designs, our method normalizes all reward values, mapping them to the same range :

$$ATCF(tc)_i = \begin{cases} v & \text{if } tc_i \text{ is new} \\ 0 & \text{otherwise} \end{cases} + 2 \cdot TCF(tc)_i,$$

(14)

[< back](#)

$$TCF(tc)_i = \begin{cases} tc.verdict_i & \text{if } tc \in TS_i, \\ 0 & \text{otherwise,} \end{cases}$$

(15)

where i represents the i th CI cycle, $tc.verdict_i$ indicates the execution result of test case tc , with 0 indicates tc passed and 1 indicates tc . v represents the additional reward value. In our study, the setting of v is consistent with the original research, where $v = 1$:

$$APHF_i(tc) = 1 - \frac{\sum_{j=1}^m Rank_j}{n \cdot m} + \frac{1}{2 \cdot n},$$

(16)

where i represents the i th CI cycle, n denotes the historical execution count of test case tc , m signifies the total number of failures of test case tc in n executions, and $Rank_j$ indicates the inverse order of the j th most recent failure of test case tc in its historical executions, and $APHF \in (0, 1)$.

Hyperparameters: Table 3 shows some of the adjustable hyperparameters of the proposed method, which were mainly set according to the recommendations of previous studies and the same parameter settings were fixed and used in this experiment [4, 39]. For the baseline method, the hyperparameters were kept the same as the original study [7].

Table 3. Hyperparameter settings.

Hyperparameter	Value
CI cycle's time limit M	$50\% \cdot TS.duration$
Hidden size for agent model	16
Experience replay memory size	10,000
Experience replay batch size	1000
Additional experience replay memory size	10,000
Additional experience replay batch size	1000
Current observation window w_c	5
Historical observation window w_h	7

[< back](#)

Learning rate η	0.02
Statistical test threshold ϵ	0.05

Environment: All experiments are conducted on a Dell tower server with 1000G RAM, two Intel Xeon Silver 4316 CPUs at 2.30 GHz and one 24G GeForce RTX 4090 D GPU, running Ubuntu 20.04.6 LTS. The agent of the RL framework in the proposed method uses the Long Short Term Memory neural network, followed by a fully connected layer at the end to output the prioritization scores of test cases.

5. Results Analysis

5.1. RQ1: Performance of EAA in terms of prioritization effectiveness and fluctuation mitigation

In this RQ, we compare EAA with three other RL-TCP approaches: RETECS, FoNT and RL-REC. By conducting this comparison, we evaluate the prioritization performance of the proposed method (Table 4, Table 5).

Table 4. Performance of EAA and other three methods in terms of NAPFD, Re

Dataset	NAPFD				Recall					
	Retecs	FoNT	RL-REC	EAA	Retecs	FoNT	RL-REC	EAA	Retecs	Fo
Apache Commons	0.1971	0.3971	0.4061	0.4612	0.9048	0.9718	0.5721	0.6717	377.344	296.1
Apache Drill	0.3193	0.5599	0.4430	0.6321	0.6511	0.7447	0.6948	0.796	11.5038	4.6
Apache Parquet	0.2087	0.612	0.4437	0.622	0.8615	0.9378	0.6877	0.6911	86.4135	35.4
Apache Tajo	0.2865	0.3982	0.3854	0.4652	0.7268	0.8302	0.5612	0.6465	87.4482	103.1
Dspace	0.3006	0.5261	0.3908	0.5268	0.5999	0.8317	0.5719	0.6414	19.7148	15.1

[< back](#)

Dataset	Retecs	FoNT	RL- REC	EAA	Retecs	FoNT	RL- REC	EAA	Retecs	Fo
Google Auto	0.1493	0.6353	0.3912	0.6477	0.4635	0.7381	0.5921	0.7167	10.3032	4.70
Google Closure	0.1983	0.4687	0.3837	0.5772	0.6182	0.783	0.5433	0.6553	113.5408	128.0
Google Guava	0.7823	0.8583	0.3873	0.8273	0.9560	0.9826	0.5288	0.8746	122.4892	89.9
IOF/ROL	0.2398	0.3936	0.3043	0.4034	0.3327	0.4911	0.4103	0.4995	6.9306	2.4
Mybatis	0.7567	0.8191	0.3898	0.8336	0.9174	0.941	0.5313	0.8731	96.6873	66.5
Paint Control	0.5371	0.6542	0.3771	0.6902	0.6424	0.7513	0.5107	0.7694	5.0056	5.80
Rails	0.3402	0.3833	0.3920	0.5833	0.4768	0.494	0.5603	0.708	43.8423	82.2
Average	0.3597	0.5588	0.3912	0.6058	0.6793	0.7914	0.5637	0.7119	81.7686	69.6

Table 5. Performance of EAA in terms of NAPFD, Recall, TTF and Duration across 12 datasets.

Dataset	NAPFD	Recall	TTF	Duration (s)
Apache Commons	0.4612	0.6717	147.0929	0.6412
Apache Drill	0.6321	0.796	3.8808	0.0813
Apache Parquet	0.622	0.6911	17.4703	0.1868
Apache Tajo	0.4652	0.6465	52.1095	0.2517
Dspace	0.5268	0.6414	5.6125	0.1463
Google Auto	0.6477	0.7167	2.9635	0.1309
Google Closure	0.5772	0.6553	40.3468	0.3299
Google Guava	0.8273	0.8746	50.4087	0.9674
IOF/ROL	0.4034	0.4995	2.2654	0.1732
Mybatis	0.8336	0.8731	29.163	0.7197
Paint Control	0.6902	0.7694	4.2721	0.1243
Rails	0.5833	0.708	52.6882	0.3464
Average	0.6058	0.7119	33.7799	0.3416

[< back](#)

representative baselines across 12 datasets in terms of NAPFD, Recall, TTF and Duration. The darker section highlights the best results among these methods. EAA demonstrates significantly better performance than the other three methods in terms of both NAPFD and TTF metrics, and also exhibits competitive results with respect to the Recall metric.

Specifically, EAA achieves the highest average NAPFD (0.6058), surpassing FoNT (0.5588), RL-REC (0.3912) and RETECS (0.3597). This demonstrates the superiority of EAA in effectively ranking test cases to detect faults earlier. Moreover, EAA achieves the best NAPFD performance on 11 out of 12 datasets (excluding Google Guava), showing a clear improvement over the baseline methods. This demonstrates the robustness and generalizability of EAA across diverse CI environments.

In terms of Recall, which reflects the fault coverage of selected test cases, EAA again exhibits competitive performance with an average of 0.7119, closely following FoNT (0.7914) and outperforming both RL-REC (0.5637) and Retecs (0.6793). Although FoNT attains slightly higher Recall, it underperforms in NAPFD and TTF, suggesting that its broader fault coverage comes at the cost of delayed fault detection. In contrast, EAA strikes a better balance between detection breadth and prioritization quality.

Regarding TTF, EAA also achieves the best average performance (33.7799), advancing the fault detection by 35–50 positions compared to the other three methods. Moreover, EAA attains the best TTF results on 11 out of 12 datasets. This indicates that EAA is capable of detecting the first fault significantly earlier than the competing approaches. This is particularly critical in CI scenarios where early feedback is essential. While EAA incurs higher training time (0.3416 s on average), in actual production environments, as long as the Duration does not exceed 1 s, it will not have a significant impact on overall production efficiency. In other words, the less than one second time overhead introduced by EAA is considered acceptable in practical scenarios. Given the substantial improvements EAA demonstrates in fault localization effectiveness, the marginal increase in execution time is considered acceptable in practical applications.

To ensure that the observed improvements achieved by the EAA method were not accidental, we calculated the variance across 30 runs and conducted the Wilcoxon rank-sum test to assess statistical significance against the baseline methods on the NAPFD metric ($p < 0.05$). We

[< back](#)

effectiveness.

As shown in the Table 6, EAA yields p -values greater than 0.05 only when compared with FoNT on Dspace and Google Guava. On all other datasets, it demonstrates statistically significant superiority over the baseline methods. In addition, the variance of EAA over 30 independent runs for each dataset is almost negligible, which underscores the stability and reproducibility of its performance.

Table 6. Wilcoxon rank-sum test results for EAA versus baseline methods on the NAPFD metric.

Dataset	RETECS	FoNT	RL-REC	Variance (EAA)
Apache Commons	1.51×10^{-11}	3.86×10^{-2}	2.32×10^{-5}	1.66×10^{-3}
Apache Drill	1.51×10^{-11}	9.28×10^{-10}	1.67×10^{-11}	1.06×10^{-3}
Apache Parquet	1.51×10^{-11}	1.75×10^{-3}	1.51×10^{-11}	3.84×10^{-4}
Apache Tajo	1.51×10^{-11}	1.51×10^{-11}	1.51×10^{-11}	6.10×10^{-5}
Dspace	1.51×10^{-11}	2.23×10^{-1}	1.51×10^{-11}	7.40×10^{-5}
Google Auto	6.21×10^{-12}	1.02×10^{-5}	1.84×10^{-11}	1.12×10^{-3}
Google Closure	1.51×10^{-11}	1.51×10^{-11}	1.51×10^{-11}	4.56×10^{-4}
Google Guava	1.34×10^{-6}	2.32×10^{-1}	1.51×10^{-11}	4.30×10^{-4}
IOF/ROL	1.51×10^{-11}	2.03×10^{-2}	1.51×10^{-11}	9.40×10^{-5}
Mybatis	1.51×10^{-11}	5.53×10^{-5}	1.51×10^{-11}	1.54×10^{-4}
Paint Control	6.23×10^{-12}	4.05×10^{-10}	1.51×10^{-11}	7.70×10^{-5}
Rails	6.23×10^{-12}	1.51×10^{-11}	1.51×10^{-11}	2.80×10^{-5}

In particular, we recorded occurrences where the NAPFD metric dropped to zero during TCP process. A NAPFD value of 0 typically occurs under two scenarios: one is when no failed test cases are positioned within the top 50% of the execution time during a cycle; the other is when the only failed test case in the cycle is placed at the very end of the top 50% of execution time. Both situations indicate significant deficiencies in the prioritization strategy and reflect severe performance fluctuations. In practice, given that there are comparatively limited instances where only a single test case fails, the majority of occurrences where NAPFD drops to zero

[< back](#)

the prioritization strategy.

Table 7 reports the number of CI cycles in which each method achieved a NAPFD score of 0 across 12 datasets. Overall, EAA significantly reduces the number of zero-score cycles compared to RETECS and FoNT. On average, EAA produces only 8.417 zero-score cycles, which is nearly 90% fewer than RETECS (79.50) and 87% fewer than FoNT (66.25).

Interestingly, the RL-REC method exhibits the fewest instances of NAPFD dropping to zero. This suggests that leveraging rich historical features of test case executions is beneficial for achieving a more stable prioritization strategy. Although RL-REC shows slightly fewer zero-score cycles (7.50) on average, its performance on other core metrics (as shown in Table 4) is less competitive, indicating that EAA offers a better overall trade-off.

Table 7. Number of cycles with NAPFD = 0 for EAA and other three methods across 12 datasets.

Dataset	RETECS	FoNT	RL-REC	EAA
Apache Commons	2	0	0	0
Apache Drill	2	1	1	1
Apache Parquet	3	2	0	0
Apache Tajo	102	82	3	9
Dspace	30	20	0	5
Google Auto	7	5	2	4
Google Closure	28	18	2	2
Google Guava	0	0	0	0
IOF/ROL	65	54	53	53
Mybatis	0	0	0	0
Paint Control	38	4	4	4
Rails	677	609	25	23
Average	79.50	66.250	7.50	8.417

Moreover, EAA demonstrates significant improvements on the Apache Tajo and Rails datasets. In particular, on the Rails dataset, EAA reduces the metric value by approximately 600

[< back](#)

characterized by a high number of test cases and frequent testing cycles. Compared to other datasets, RETECS and FoNT methods perform relatively poorly on them, suggesting that these two datasets, to some extent, better reflect the complexity of environmental changes in CI testing. EAA demonstrates strong performance on these two datasets, further confirming its ability to effectively adapt to complex changes in CI environments and to mitigate performance.

FoNT's outstanding performance in the Recall metric indicates that the combined use of predefined rules and the continuous assignment of additional rewards effectively detect more failing test cases. Nevertheless, FoNT still has numerous instances in which the NAPFD metric drops to zero, partly indicating that the agent's excessive focus on new test cases leads to prioritization strategy failures, i.e. the agent's excessive dependence on the guidance of the additional rewards leads to inappropriate adjustments of the prioritization strategy.

Furthermore, both RETECS and RL-REC adopt a periodic experience replay strategy, which fails to respond promptly to environmental changes, resulting in relatively modest prioritization performance.

By detecting changes in the CI, EAA selectively adds additional rewards to test cases. During training, it further constrains gradient updates through a reconciliation method. In this way, EAA makes timely use of the guidance provided by additional rewards, while avoiding excessive adjustments to the prioritization strategy. As a result, both new and previously learned prioritization knowledge are effectively incorporated. Consequently, EAA responds promptly to environmental changes and incorporates the signals reflecting such changes into the agent's training process. It significantly reduces occurrences of NAPFD falling to zero, improves both NAPFD and TTF metrics, effectively mitigates prioritization performance fluctuations, and improves the overall TCP performance.

5.2. RQ2: Effectiveness of each component in EAA

In this RQ, we investigate the impact of each component within the EAA method framework. EAA represents the complete method that incorporates both the reconciliation component and the environment change detection with additional reward triggering component. Compared to EAA, EAA_{R-} excludes the reconciliation component, while EAA_{E-R-} omits both the

[< back](#)

triggering component.

Table 8 summarizes the performance of these methods across four metrics: NAPFD, Recall, TTF and Duration. The darker section highlights the best results. It can be observed that as more components are incorporated, the overall prioritization performance of the method shows an upward trend.

Table 8. Performance of EAA and the other two variant versions in terms of NAPFD, Recall, across 12 datasets.

Dataset	NAPFD			Recall			TTF			EA
	EAA	EAA	EAA	EAA	EAA	EAA	EAA	EAA _{R-}	EAA	
	<i>E⁻R⁻</i>	<i>R⁻</i>		<i>E⁻R⁻</i>	<i>R⁻</i>		<i>E⁻R⁻</i>			<i>E⁻</i>
Apache Commons	0.487	0.4437	0.4612	0.502	0.5828	0.6717	52.285	104.6905	147.0929	0.5
Apache Drill	0.6223	0.6221	0.6321	0.7896	0.7858	0.796	4.0269	4.0833	3.8808	0.06
Apache Parquet	0.5873	0.6	0.622	0.6703	0.6683	0.6911	15.4156	17.2165	17.4703	0.14
Apache Tajo	0.3740	0.4338	0.4652	0.5384	0.605	0.6465	58.5751	54.2464	52.1095	0.1
Dspace	0.4817	0.5134	0.5268	0.6452	0.6131	0.6414	10.006	5.5653	5.6125	0.0
Google Auto	0.5894	0.6457	0.6477	0.6548	0.7008	0.7167	3.5095	2.9365	2.9635	0.02
Google Closure	0.4861	0.5244	0.5772	0.6405	0.5743	0.6553	88.8266	42.6524	40.3468	0.28
Google Guava	0.8481	0.8104	0.8273	0.8594	0.8674	0.8746	14.1877	61.7421	50.4087	0.92
IOF/ROL	0.4056	0.4047	0.4034	0.5024	0.5004	0.4995	2.2701	2.2055	2.2654	0.09
Mybatis	0.8159	0.8085	0.8336	0.8464	0.8395	0.8731	26.9794	27.9485	29.163	0.79
Paint	0.6828	0.6885	0.6902	0.7561	0.7635	0.7694	4.4152	4.2182	4.2721	0.0
Control										

[< back](#)

Dataset	EAA _{E⁻R⁻}	EAA _{R⁻}	EAA	EAA _{E⁻R⁻}	EAA _{R⁻}	EAA	EAA _{E⁻R⁻}	EAA _{R⁻}	EAA	EAA _{E⁻R⁻}
Rails	0.5321	0.5336	0.5833	0.6771	0.6782	0.708	68.0257	67.6361	52.6882	0.32
Average	0.5760	0.5857	0.6058	0.6735	0.6816	0.7119	29.0436	32.9284	33.7799	0.29

Specifically, EAA_{E⁻R⁻}, which does not incorporate any additional components, demonstrates slight advantages in TTF and Duration. However, it performs poorly overall in terms of NAPFD and Recall. With the introduction of the environment change detection and additional reward triggering mechanism, EAA_{R⁻} shows marginal improvements in both NAPFD and Recall. Compared to EAA_{E⁻R⁻}, EAA_{R⁻} achieves improvements of 0.97% in NAPFD and 0.81% in Recall, respectively, despite incurring a delay of 3.9 positions in TTF and a slight increase in Duration. Although these improvements in NAPFD and Recall are minor, the results presented in Table 6 confirm the effectiveness of this component in promptly responding to environmental changes and enhancing prioritization stability. EAA_{R⁻} significantly reduces the occurrence of NAPFD = 0, which indicates extremely unstable prioritization performance, but the overall improvement in prioritization effectiveness remains limited. This is because, despite the environment change detection and additional reward triggering components being more adaptive to environmental variations, they may interfere with the previously learned prioritization knowledge during the agent's strategy update process.

With the subsequent introduction of the reconciliation component, EAA achieves a more notable improvement in performance, indicating that this component effectively integrates newly acquired and previously learned prioritization knowledge. Compared to EAA_{R⁻}, EAA shows substantial gains in NAPFD and Recall, with increases of 2.01% and 3.03%, respectively. Moreover, EAA achieves the best NAPFD and Recall performance across the vast majority of datasets. Although TTF increases by approximately 0.85 positions, this is primarily attributed to a significant increase in the Apache Commons dataset, while the TTF performance on most other datasets remains comparable or even superior. The application of the reconciliation component also results in a slight increase in Duration 0.0432 seconds more than EAA_{R⁻}. Considering the overall performance improvements, this overhead is deemed acceptable.

[< back](#)

method and its two variants. The base variant $EAA_{E^-R^-}$, which does not include any components, exhibits a high number of NAPFD=0 cycles across most datasets, indicating significant instability. By introducing environment change detection and additional reward triggering components, EAA_{R^-} significantly reduces instances of highly unstable prioritization performance across nearly all datasets, decreasing the average count from 56.667 to 13.25. Notably, EAA_{R^-} reduces the count from 203 to 22 on Apache Tajo and from 316 to 23 on Rails. The performance on these larger and more complex datasets further underscores the component's ability to adapt promptly to environmental changes and their effectiveness in mitigating prioritization performance fluctuations.

Table 9. Number of cycles with NAPFD = 0 for EAA and other two variant versions across 12 datasets.

Dataset	$EAA_{E^-R^-}$	EAA_{R^-}	EAA
Apache Commons	9	2	0
Apache Drill	1	1	1
Apache Parquet	18	7	0
Apache Tajo	203	22	9
Dspace	34	29	5
Google Auto	4	3	4
Google Closure	17	11	2
Google Guava	5	1	0
IOF/ROL	58	53	53
Mybatis	2	2	0
Paintcontrol	13	5	4
Rails	316	23	23
Average	56.667	13.25	8.417

The complete EAA method further integrates the reconciliation method, enabling better fusion of previously learned knowledge with new prioritization knowledge that reflects environmental changes, thereby demonstrating the best overall performance. It not only continues the trend of reducing CI cycles with NAPFD = 0 but also completely eliminates such cycles in several

[< back](#)

number of NAPFD = 0 cycles is further reduced to 8.417, confirming the cumulative benefits of all components.

The overall trend supports the conclusion that each added component makes a meaningful contribution to improving prioritization performance and enhancing prioritization stability.

6. Threats to Validity

This section provides a comprehensive analysis of the possible threats to the study from four key perspectives: internal validity, external validity and construct validity. For each aspect, we discuss the main concerns and the mitigation strategies adopted to address them, aiming to offer valuable insights for future research.

Internal Validity: The primary threat comes from the potential impact of randomness during the training process of the RL model. To mitigate this threat, we repeated the experiment 30 times and used the average value as the final result. In the field of machine learning, parameter settings are sensitive to various environmental conditions, yet our study did not optimize parameters for different environments.

External Validity: The main threat arises from the datasets used. To better simulate the actual CI TCP environment, we conducted experiments on 12 real-world industrial datasets of varying scales. Each dataset contains thousands of CI cycles and hundreds of test cases on average, which helps alleviate the external validity threat.

Construct Validity: Construct threats mainly relate to the choice of evaluation metrics. To comprehensively assess the method's effectiveness, we used four metrics — NAPFD, TTF, Recall and Duration — in the experiments. These metrics are widely used in the field of CI TCP and can fully reflect the method's performance across various aspects.

7. Conclusion

We introduce a novel method, EAA, which incorporates environmental change factors into the agent retraining process. Specifically, EAA detects CI environment changes and assigns additional rewards to test cases to reflect environment changes when significant changes occur. During the agent retraining phase, a reconciliation method is employed to prevent excessive

[< back](#)

learned knowledge. EAA effectively mitigates performance fluctuations in TCP and enhances overall prioritization performance. We conducted experimental evaluations on 12 industrial datasets. The results show that EAA exhibits superior TCP performance compared to other RL-TCP method. Specifically, EAA achieves improvements of 4.7–24.79% in the NAPFD metric and advances the TTF metric by 35.85–50.37 positions. EAA significantly reduces the number of CI cycles with NAPFD equal to zero. Furthermore, the effectiveness of each component within EAA is validated through experimental comparisons. Overall, EAA demonstrates that incorporating environment change detection and gradient projection into RL-TCP significantly improves stability and fault detection, highlighting its strong potential for industrial CI pipelines.

8. Future Work

While the proposed EAA method shows improvements in mitigating prioritization instability and enhancing fault detection within CI pipelines, several avenues for future research remain open.

First, scalability to larger CI pipelines is an important challenge. Modern DevOps environments often involve thousands of test cases and highly complex dependency structures. Extending EAA to handle such large-scale, heterogeneous pipelines efficiently will further enhance its industrial applicability. Second, integration with CI/CD toolchains and DevOps practices should be explored. Embedding EAA into mainstream platforms such as Jenkins, GitLab CI or GitHub Actions may introduce additional constraints and performance trade-offs that require further investigation. Third, explainability of RL policies is increasingly critical in industrial applications. Providing interpretable rationales for prioritization decisions would help developers build trust in RL-TCP techniques and facilitate debugging in practice. Techniques from explainable AI (XAI) could be adapted to shed light on how EAA reacts to environmental changes. Finally, investigating hybrid approaches that combine RL with other optimization paradigms, such as graph learning, or large language models, may yield richer prioritization policies. In particular, LLMs with retrieval-augmented reasoning could enhance test case representation and facilitate cross-project generalization.

Acknowledgments

[< back](#)

of China, with Grant Numbers 61872026, 61902015 and 62077003.

ORCID

Zheng Li  <https://orcid.org/0000-0002-3938-7033>

Jiping Liu  <https://orcid.org/0009-0005-5187-4853>

Shunqing Xu  <https://orcid.org/0009-0002-2030-2228>

Hengyuan Liu  <https://orcid.org/0000-0002-5884-2089>

Yong Liu  <https://orcid.org/0000-0003-1754-3039>

Note

^a <https://github.com/1394827282/EAA/>

We recommend

Deep Q-Network with Reinforcement Learning for Fault Detection in Cyber-Physical Systems

J. Stanly Jayaprakash, Journal of Circuits, Systems and Computers, 2022

A Hybrid Online Off-Policy Reinforcement Learning Agent Framework Supported by Transformers

Enrique Adrian Villarrubia-Martin, International Journal of Neural Systems, 2023

Neuro-Inspired Reinforcement Learning to Improve Trajectory Prediction in Reward-Guided Behavior

Bo-Wei Chen, International Journal of Neural Systems, 2022

REINFORCEMENT LEARNING IN SUPPLY CHAINS

ANNAPURNA VALLURI, International Journal of Neural Systems, 2011

CODE-SPECIFIC LEARNING RULES IMPROVE ACTION SELECTION BY POPULATIONS OF SPIKING NEURONS

JOHANNES FRIEDRICH, International Journal of Neural Systems, 2014

Overview and prospects of modeling and optimal control for non-ferrous metallurgical processes and mineral processing [↗](#)

Green and Smart Mining Engineering, 2025

AI dermatology: Reviewing the frontiers of skin cancer detection technologies [↗](#)

Zhengyu Yu, Intelligent Oncology , 2025

A Critical Analysis of the Readiness of Institutions of Higher Education to Enhancing Learning Through the Integration of E-Learning [↗](#)

Mfundo Mandla Masuku, Journal of Educational Studies, 2022

Small Changes Have Big Effects: Non-Structural Sound Absorbing Furnishings and Reverberation Time in a Preschool Classroom [↗](#)

Victor de Andrade, Journal of Educational Studies, 2023

Artificial intelligence in shale gas and oil: A comprehensive review of applications and challenges [↗](#)

Green and Smart Mining Engineering, 2025

[< back](#)



Privacy policy

© 2026 World Scientific Publishing Co Pte Ltd

Powered by Atypon® Literatum